

ECOCIVIX AI

05 — AI Intelligence Specification

TEAM VELOCITY • HackYatra × EDM 3.0 • PRE-HACKATHON PLANNING

Purpose: Define the intelligence layer that converts citizen reports into structured, explainable and validated civic, environmental and safety insights.

Status: PRE-HACKATHON DRAFT / PLANNING SPECIFICATION

Approval authority: Garv Nain

AI Objective

- Answer five product questions: What is happening? How important is it? Who should act? What impact does it have? What should happen next?

Input

- Issue title; citizen description; optional image; location/context where available.
- The AI should receive only necessary data and should not be given secrets or private credentials.

Core Output

- category; priority; summary; department; environmentalImpact; environmentalReason; safetyRisk; safetyReason; suggestedAction.
- The exact API schema must be finalized in Document 08 before implementation.

Intelligence Pipeline

- Receive report → normalize input → AI classification → impact assessment → routing suggestion → structured parsing → schema validation → allowed-value validation → business-rule validation → persistence.

Civic Intelligence

- Classify issues such as roads, streetlights, infrastructure, water/electricity, sanitation and public facilities.
- Suggest responsible department and practical next action.

Eco Intelligence

- Identify waste, water leakage/wastage, pollution, cleanliness and energy-wastage relevance.
- Avoid unsupported exact claims such as CO2 savings without defensible calculations or data sources.

Safety Intelligence

- Identify safety relevance of civic conditions and safety-related reports.
- Never claim guaranteed protection, emergency response, crime prediction or perfect threat detection.

Confidence & Explainability

- Where supported, show concise reasons for classifications rather than hidden reasoning.
- Use human-readable evidence from the report: e.g., “Broken streetlight near hostel” supports high safety relevance as a civic-condition assessment.

Reliability

- Handle malformed JSON, invalid categories, missing fields, timeout, provider failure and rate limits.
- Never label fallback output as Gemini output.
- Fallback behavior must be explicit and deterministic where used.

Prompting

- Prompts should define role, input schema, output schema, allowed values, safety constraints and refusal/uncertainty behavior.
- Prompts must not instruct the model to fabricate facts or exact impact metrics.

Validation

- Use schema validation; enum validation; length checks; required fields; safe text handling; business rules.
- AI output is not trusted merely because it parses as JSON.

Human Oversight

- AI recommends. Administrators can review, correct, assign and update issues.
- Administrative decisions must remain visibly separate from AI suggestions.

AI Failure UX

- Show that AI analysis is unavailable when it fails.
- If a rule-based fallback exists, label it as fallback.
- Core issue reporting should remain recoverable where feasible.

AI Acceptance Criteria

- Structured output; deterministic validation; failure handling; server-side secrets; distinguish AI from human decisions; real data in final demo; no unsupported safety/environmental claims.